



Technologies Python

Simple · Polyvalent · Le Langage du Futur

#1

Langage le plus populaire

15M+

Développeurs mondiaux

10+

Domaines d'application

30 ans

D'évolution continue

Pourquoi Python ?



Syntaxe lisible

Proche du langage naturel. Apprendre Python en semaines, pas en années.



Productivité maximale

10x moins de code que Java/C++. Développez plus vite, livrez plus tôt.



Écosystème immense

400 000+ packages sur PyPI. Une bibliothèque pour chaque besoin.



Communauté mondiale

15M+ développeurs, des forums actifs, des ressources gratuites infinies.



Marché du travail

Parmi les compétences les mieux rémunérées. Demande en forte croissance.



Multiplateforme

Windows, Linux, macOS, cloud, embarqué. Fonctionne partout.

La Philosophie Python — Le Zen de Python

import this

"Beautiful is better than ugly."

"Explicit is better than implicit."

"Simple is better than complex."

"Readability counts."

"There should be one obvious way to do it."

Python vs autres langages

Java

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello!");  
    }  
}
```

Python

```
print("Hello!")
```

**Python = même résultat
en 10x moins de code**

Python de Base — Étape 1 : Variables & Types

```
# Variables – pas de déclaration de type !
nom = "Alice"           # str (chaîne)
age = 25                 # int (entier)
taille = 1.68            # float (décimal)
actif = True             # bool (booléen)
scores = [90, 85, 78]    # list (liste)
info = {"ville": "Tana"}# dict (dictionnaire)

# Python détecte automatiquement le type
print(type(nom))         # <class 'str'>
print(type(scores))      # <class 'list'>

# Affichage formaté (f-string)
print(f"{nom} a {age} ans et mesure {taille}m")
# → Alice a 25 ans et mesure 1.68m
```

Typage dynamique

Python devine le type automatiquement. Pas de `int x = 0;` comme en Java.

Listes puissantes

Une liste peut contenir n'importe quel type. `[1, 'deux', 3.0, True]`

Dictionnaires

Clé → Valeur. Structure de données ultra-utilisée en Python.

f-strings

Interpolation moderne : `f"Bonjour {nom}"`. Clair et lisible.

Python de Base — Étape 2 : Conditions & Boucles

```
# — CONDITIONS —————
note = 85

if note >= 90:
    print("Excellent !")
elif note >= 70:
    print("Bien !")      # ← ce bloc s'exécute
else:
    print("À améliorer")

# — BOUCLE for —————
fruits = ["mango", "litchi", "avocat"]
for fruit in fruits:
    print(f" → {fruit}")

# — BOUCLE while —————
compteur = 0
while compteur < 3:
    print(f"Tour {compteur}")
    compteur += 1

# — LIST COMPREHENSION (Python unique !) —
carres = [x**2 for x in range(5)]
# → [0, 1, 4, 9, 16]
```

Indentation = Structure

Pas d'accolades { }. L'indentation (4 espaces) définit les blocs.

for ... in ...

Itère sur n'importe quel objet : liste, fichier, dictionnaire, range...

List Comprehension

Syntaxe compacte pour créer des listes. En une ligne ce qui prendrait 4 lignes ailleurs.

Pas de ; requis

Une instruction par ligne. Pas de point-virgule obligatoire.

Python de Base — Étape 3 : Fonctions & Classes

```
# — FONCTION —
def saluer(nom, langue="fr"):
    if langue == "mg":
        return f"Manao ahoana, {nom} !"
    return f"Bonjour, {nom} !"

print(saluer("Fontaine"))      # Bonjour, Fontaine !
print(saluer("Rakoto", "mg"))  # Manao ahoana, Rakoto !

# — CLASSE (Programmation Orientée Objet) —
class Patient:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age

    def info(self):
        return f"{self.nom}, {self.age} ans"

# Utilisation
p = Patient("Marie", 30)
print(p.info())  # Marie, 30 ans
```

1 def — définir

Le mot-clé `def` crée une fonction. Paramètres optionnels avec valeurs par défaut.

2 return — retourner

Retourne un résultat. Une fonction peut retourner plusieurs valeurs simultanément.

3 class — créer

Regrouper données + comportements. `__init__` est le constructeur.

4 self — référencer

Référence à l'objet lui-même. Accède aux attributs de l'instance.

Python de Base — Ce que Python sait faire SEUL

La bibliothèque standard Python : 200+ modules inclus

os / sys

Gestion fichiers, dossiers, variables d'environnement

```
os.listdir('/data')
```

datetime

Dates, heures, calculs temporels

```
datetime.now()
```

re

Expressions régulières
(recherche/remplacement)

```
re.findall(r'\d+', texte)
```

csv

Lecture/écriture de fichiers CSV

```
csv.reader(open('data.csv'))
```

json

Lire/écriture des données JSON

```
json.dumps({'key': 'val'})
```

math

Fonctions mathématiques avancées

```
math.sqrt(144) → 12.0
```

urllib

Requêtes HTTP sans dépendance externe

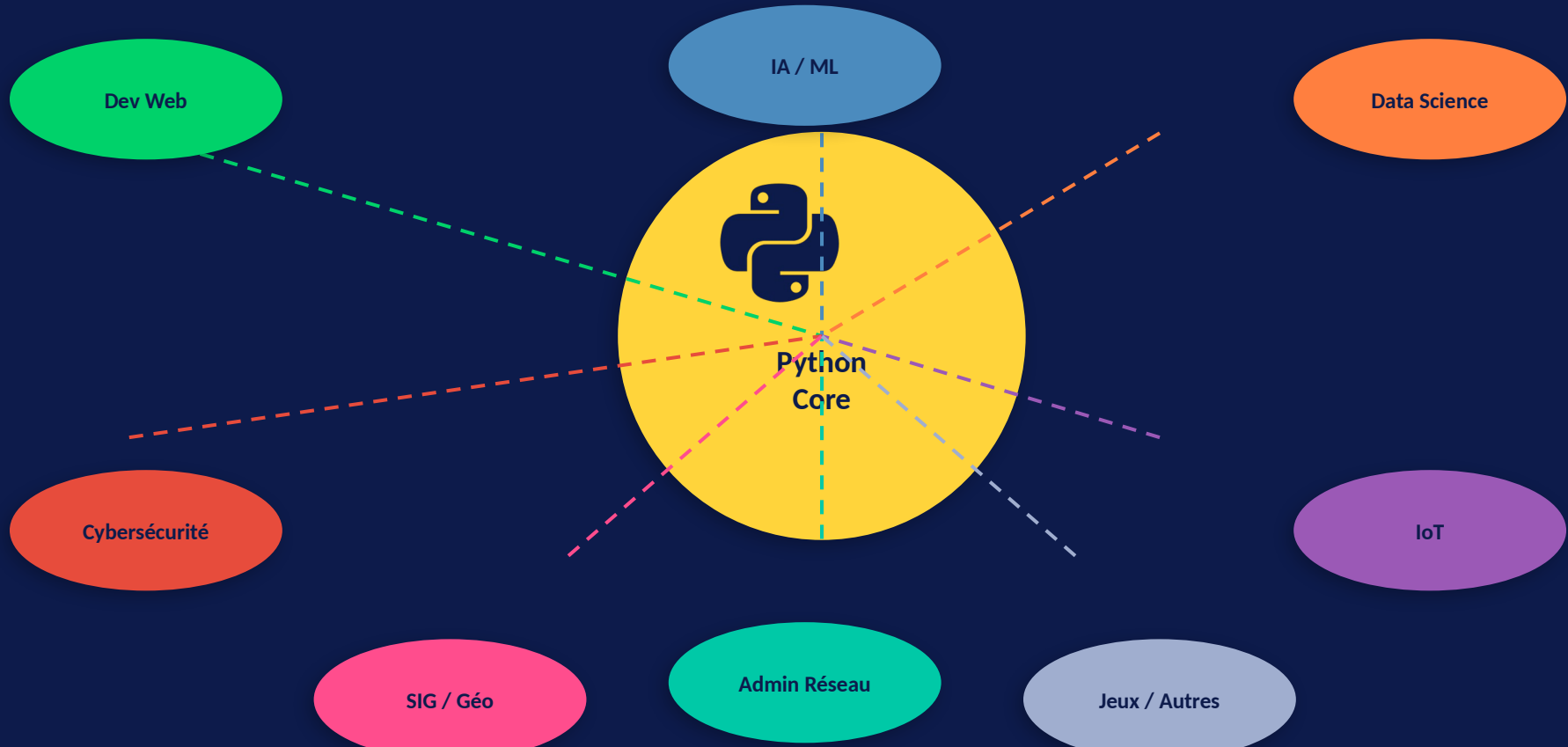
```
urllib.request.urlopen(url)
```

threading

Exécution parallèle multi-thread

```
threading.Thread(target=fn)
```

Le Principe Clé — Python Core + Framework = Spécialiste



Python seul = généraliste | Python + Framework spécialisé = EXPERT du domaine



Django

Fullstack

Utilisé par : Instagram, Disqus, Mozilla

Framework complet «batteries incluses». ORM, admin auto, auth, templates. Idéal pour applications web complexes.



Flask

Micro

Utilisé par : Pinterest, LinkedIn, Netflix

Minimaliste et flexible. Vous choisissez chaque composant. Parfait pour APIs, microservices et prototypes rapides.



FastAPI

API Moderne

Utilisé par : Uber, Microsoft, Netflix

Le plus rapide des frameworks Python. Basé sur Python 3.8+, documentation auto Swagger, validation avec Pydantic.



Développement Web — De 0 à une API REST en 3 étapes

1 Route simple (Flask)

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def accueil():
    return "Bienvenue sur mon API !"

app.run(debug=True)
```

2 Endpoint JSON

```
from flask import Flask, jsonify

@app.route("/patients")
def liste_patients():
    patients = [
        {"id":1, "nom":"Rakoto",
        "age":35},
        {"id":2, "nom":"Vola",
        "age":28},
    ]
    return jsonify(patients)
```

3 API Complète avec FastAPI

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Patient(BaseModel):
    nom: str; age: int

@app.post("/patients")
def creer(p: Patient):
    return {"message": f"{p.nom}
    créé !"}
```

pip install flask fastapi uvicorn → Serveur web opérationnel en 5 lignes de Python !



Intelligence Artificielle & ML — Écosystème



Scikit-learn

ML classique : classification, régression, clustering. Interface unifiée `.fit()/.predict()`. Idéal pour débuter en ML.

Cas d'usage : spam, fraude, diagnostic

TensorFlow / Keras

Deep learning par Google. Keras : API haut niveau pour réseaux de neurones. Déploiement mobile et web (TF.js).

Cas d'usage : vision, NLP, série temporelle

PyTorch

Deep learning par Meta. Favori de la recherche académique. Graphe dynamique, débogage facile.

Cas d'usage : recherche, LLM, génératif

HuggingFace 🤗

La bibliothèque des LLMs. 200 000+ modèles pré-entraînés. Traduction, résumé, NLP, génération de texte.

Cas d'usage : chatbots, traduction, NLP



IA & ML — Pipeline Complet en 5 Étapes

```
# — PIPELINE ML COMPLET EN PYTHON —  
# Étape 1 : Charger les données  
from sklearn.datasets import load_breast_cancer  
from sklearn.model_selection import train_test_split  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.metrics import accuracy_score  
  
data = load_breast_cancer()  
X, y = data.data, data.target  
  
# Étape 2 : Diviser train/test  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)  
  
# Étape 3 : Choisir et entraîner le modèle  
model = RandomForestClassifier(n_estimators=100)  
model.fit(X_train, y_train)  
  
# Étape 4 : Prédire  
predictions = model.predict(X_test)  
  
# Étape 5 : Évaluer  
accuracy = accuracy_score(y_test, predictions)  
print(f"Précision : {accuracy:.2%}") # → Précision : 97.37%
```

1 Charger

2 Diviser

3 Entraîner

4 Prédire

5 Évaluer



Pandas

Manipulation de données tabulaires. DataFrames puissants. Lecture CSV, Excel, SQL, JSON. Indispensable en Data Science.

```
pip install pandas
```

NumPy

Calcul numérique haute performance. Tableaux multidimensionnels, algèbre linéaire, transformée de Fourier.

```
pip install numpy
```

Matplotlib / Seaborn

Visualisation de données. Matplotlib pour personnalisation totale, Seaborn pour graphiques statistiques élégants.

```
pip install matplotlib seaborn
```

Jupyter Notebook

Environnement interactif. Mélange code + graphiques + texte dans un seul document. Standard en Data Science.

```
pip install jupyter
```



Data Science — Du CSV à la Visualisation en 4 Étapes

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Étape 1 : Charger et explorer
df = pd.read_csv("patients.csv")
print(df.head())      # 5 premières lignes
print(df.describe())   # statistiques descriptives

# Étape 2 : Nettoyer les données
df = df.dropna()        # supprimer valeurs manquantes
df["age"] = df["age"].astype(int)  # convertir types
df = df[df["age"] > 0]   # filtrer lignes invalides

# Étape 3 : Analyser
groupe = df.groupby("region")["cas"].sum()
print(groupe.sort_values(ascending=False))

# Étape 4 : Visualiser
plt.figure(figsize=(10, 6))
sns.barplot(data=df, x="region", y="cas", palette="viridis")
plt.title("Cas par Région")
plt.savefig("rapport.png", dpi=150)
plt.show()
```

head()

Aperçu rapide du jeu de données

describe()

Moyenne, écart-type, min/max automatiques

groupby()

Agrégation par catégorie en 1 ligne

barplot()

Graphique professionnel en 3 lignes



Cybersécurité & Hacking Éthique — Écosystème



Usage strictement éthique et légal — Tests sur systèmes autorisés uniquement

Scapy

Réseau / Forensic

Forge et analyse des paquets réseau. Scan ARP, sniffing, traceroute personnalisé, injection de paquets.

Requests + BS4

OSINT / Recon

Reconnaissance web. Scraping de pages, extraction de liens, analyse de formulaires HTML, web crawling.

Paramiko

SSH / Infrastructure

Client SSH Python. Connexions sécurisées, transfert de fichiers SFTP, brute-force SSH (audit), tunnels.

Cryptography / Hashlib

Crypto / Chiffrement

Hachage MD5, SHA-256, bcrypt. Chiffrement AES, RSA, signatures numériques, certificats TLS.

Python-nmap

Scan / Énumération

Wrapper Python pour Nmap. Scan de ports, détection OS, fingerprinting de services réseau.

Impacket

Pentest Windows

Protocoles réseau bas niveau. SMB, NTLM, Kerberos. Utilisé dans les pentest Windows/Active Directory.

Cybersécurité — Scan de Ports & Rapport Automatique

```
import socket
import concurrent.futures

# — ÉTAPE 1 : Scanner les ports TCP —————
def scan_port(host, port):
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.settimeout(0.5)
        result = sock.connect_ex((host, port))
        sock.close()
        return port if result == 0 else None
    except: return None

# — ÉTAPE 2 : Scan parallèle (rapide) —————
def scan_host(host, port_range=(1, 1024)):
    ports_ouverts = []
    with concurrent.futures.ThreadPoolExecutor(max_workers=100) as executor:
        futures = {executor.submit(scan_port, host, p): p
                    for p in range(*port_range)}
        for f in concurrent.futures.as_completed(futures):
            if f.result(): ports_ouverts.append(f.result())
    return sorted(ports_ouverts)

# — ÉTAPE 3 : Générer rapport —————
host = "192.168.1.1"
ports = scan_host(host)
print(f"Hôte : {host}")
for p in ports:
    services = {22:"SSH", 80:"HTTP", 443:"HTTPS", 3306:"MySQL"}
    print(f"  Port {p}/TCP  OUVERT  [{services.get(p, 'inconnu')}]")
```

1

Connexion TCP

socket.connect_ex() teste l'ouverture d'un port. Timeout court = scan rapide.

2

Parallélisme

ThreadPoolExecutor lance 100 scans simultanés. 1024 ports en quelques secondes.

3

Rapport auto

Identification des services (SSH, HTTP...) et génération d'un rapport lisible.



Administration Système & Réseau — Écosystème



os / subprocess

[Scripts bash → Python]

Exécution de commandes système, gestion des processus, variables d'environnement. Fondation de tout script d'admin.

Paramiko

[SSH distant]

Client SSH complet. Connexion distante, exécution de commandes, transfert SFTP. Automatisation serveurs Linux.

Netmiko

[Routeurs / Switchs]

Automatisation d'équipements réseau (Cisco, Juniper, HP). Basé sur Paramiko, optimisé pour CLI réseau.

Fabric

[DevOps / Déploiement]

Automatisation de déploiement. Exécute des tâches sur plusieurs serveurs en parallèle via SSH.

Ansible (Python)

[Infrastructure as Code]

Outil d'automatisation IT massivement utilisé. Entièrement écrit en Python. Playbooks YAML sur du Python sous le capot.

psutil / watchdog

[Monitoring]

Surveillance système en temps réel. CPU, RAM, disque, processus. Watchdog surveille les modifications de fichiers.



Admin Réseau — Automatisation SSH Multi-Serveurs

```
import paramiko
import json
from datetime import datetime

# — ÉTAPE 1 : Configuration des serveurs —————
serveurs = [
    {"host": "192.168.1.10", "user": "admin", "name": "Web-01"},
    {"host": "192.168.1.11", "user": "admin", "name": "DB-01"},
    {"host": "192.168.1.12", "user": "admin", "name": "App-01"},
]

# — ÉTAPE 2 : Connexion SSH + Exécution de commande —————
def audit_serveur(srv):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    client.connect(srv["host"], username=srv["user"])

    # Récupérer CPU, RAM, disque
    commandes = {
        "cpu": "top -bn1 | grep 'Cpu(s)' | awk '{print $2}'",
        "ram": "free -m | awk 'NR==2{printf \"%.1f%%\\\", $3*100/$2}'",
        "disque": "df -h / | awk 'NR==2{print $5}'",
    }
    resultat = {"serveur": srv["name"], "heure": str(datetime.now())}
    for key, cmd in commandes.items():
        _, stdout, _ = client.exec_command(cmd)
        resultat[key] = stdout.read().decode().strip()
    client.close()
    return resultat

# — ÉTAPE 3 : Alertes automatiques —————
for srv in serveurs:
    data = audit_serveur(srv)
    print(f"[{data['serveur']} CPU:{data['cpu']} RAM:{data['ram']} Disque:{data['disque']}")
```

Multi-serveurs

Même script, tous vos serveurs. Boucle for = automatisation totale.

Alertes temps réel

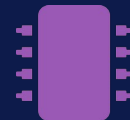
Détection automatique d'anomalies (CPU, RAM, disque).

Rapports JSON/Excel

Exporter les résultats vers JSON, CSV ou email automatiquement.



IoT & Systèmes Embarqués — Écosystème



MicroPython

Python pour microcontrôleurs (ESP32, ESP8266, STM32). Même syntaxe Python sur 256Ko de RAM. Révolutionne l'embarqué.

⚙️ ESP32 / ESP8266

CircuitPython

Fork MicroPython par Adafruit. Plug-and-play USB, bibliothèques hardware riches (capteurs, écrans, WiFi).

⚙️ Adafruit Feather / Pi Pico

RPi.GPIO / gpiozero

Contrôle des GPIO du Raspberry Pi. LEDs, boutons, capteurs, moteurs. gpiozero offre une API haut niveau intuitive.

⚙️ Raspberry Pi

MQTT (paho-mqtt)

Protocole IoT léger publish/subscribe. Connecte les capteurs au cloud (AWS IoT, MQTT broker). Standard industriel.

⚙️ Tous appareils IoT

Blynk / MQTT Dash

Dashboard mobile pour IoT. Visualisez vos données capteurs depuis un smartphone en temps réel.

⚙️ Dashboard mobile

OpenCV sur Pi

Vision par ordinateur sur Raspberry Pi. Détection de mouvement, reconnaissance de visages, QR codes.

⚙️ Raspberry Pi + Caméra



IoT — Capteur DHT11 → MQTT → Dashboard

```
# — MicroPython sur ESP32 —————
import dht, machine, network, time
from umqtt.simple import MQTTClient

# Étape 1 : Connexion WiFi
def connecter_wifi(ssid, password):
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    wlan.connect(ssid, password)
    while not wlan.isconnected():
        time.sleep(0.5)
    print("WiFi connecté :", wlan.ifconfig()[0])

# Étape 2 : Lire le capteur DHT11
capteur = dht.DHT11(machine.Pin(4))

def lire_capteur():
    capteur.measure()
    return capteur.temperature(), capteur.humidity()

# Étape 3 : Publier sur MQTT
client = MQTTClient("esp32", "broker.hivemq.com")
client.connect()

# Étape 4 : Boucle principale
connecter_wifi("MonWifi", "MonMotDePasse")
while True:
    temp, hum = lire_capteur()
    payload = f'{{"temp":{temp},"hum":{hum}}}'
    client.publish("maison/salon/meteo", payload)
    print(f"Envoyé → Temp:{temp}°C Hum:{hum}%")
    time.sleep(30) # Mesure toutes les 30 secondes
```

DHT11

Capteur Physique

ESP32

Microcontrôleur

MQTT

Protocole IoT

Dashboard

Visualisation mobile



SIG & Géospatial — Écosystème



GeoPandas

Extension de Pandas pour les données géospatiales. Shapefiles, GeoJSON, opérations spatiales (intersection, union, buffer).



Shapely

Géométrie vectorielle Python. Création et manipulation de Points, Polygones, Lignes. Calcul de distances, intersections.



Folium

Cartes interactives Leaflet.js depuis Python. Marqueurs, couches, popup, heatmaps. Export HTML interactif.



Rasterio

Lecture et traitement de données raster (images satellitaires, MNT). Reprojection, découpage, analyse bande par bande.



GDAL / Fiona

Bibliothèques fondamentales SIG. Conversion entre formats (Shapefile, GeoJSON, GeoTiff, KML, PostGIS).



PyQGIS

API Python de QGIS. Automatisation d'analyses spatiales, génération de cartes, traitements par lots dans QGIS.



SIG — Du Shapefile à la Carte Interactive en 4 Étapes

```
import geopandas as gpd
import folium
from shapely.geometry import Point

# Étape 1 : Charger un shapefile des régions de Madagascar
regions = gpd.read_file("madagascar_regions.shp")
print(regions.crs)          # Système de projection
print(regions.head())       # Aperçu des données

# Étape 2 : Analyse spatiale – Filtrer et calculer
regions["superficie_km2"] = regions.geometry.area / 1e6
analamanga = regions[regions["NOM"] == "Analamanga"]
print(f"Superficie Analamanga : {analamanga['superficie_km2'].values[0]:.0f} km²")

# Étape 3 : Ajouter des points d'intérêt (centres de santé)
centres_sante = [
    {"nom": "CSB Tana", "lat": -18.9137, "lon": 47.5362},
    {"nom": "CSB Antsirabe", "lat": -19.8659, "lon": 47.0333},
]

# Étape 4 : Générer une carte interactive HTML
carte = folium.Map(location=[-18.9, 47.5], zoom_start=6)
for c in centres_sante:
    folium.Marker(
        location=[c["lat"], c["lon"]],
        popup=c["nom"],
        icon=folium.Icon(color="red", icon="plus-sign")
    ).add_to(carte)

carte.save("centres_sante_madagascar.html")
print("Carte sauvegardée !")
```

Shapefile → DataFrame

Données géographiques comme un tableau Pandas. Filtre, tri, calcul.

Calcul de superficie

Opérations géométriques natives. Surface, périmètre, distance.

Carte HTML interactive

Export direct en carte Leaflet, zoomable, partageable.



Automatisation & Scripting — Écosystème



PyAutoGUI

[RPA / Desktop]

Contrôle souris et clavier. Automatisation d'interfaces graphiques, captures d'écran, clics automatiques.

Selenium

[Web / Tests]

Automatisation de navigateurs web. Tests fonctionnels, web scraping de pages JavaScript, remplissage formulaires.

Schedule / APScheduler

[Tâches planifiées]

Planification de tâches. Exécutez des fonctions Python à des intervalles définis (cron-like en Python pur).

Watchdog

[Surveillance fichiers]

Surveillance du système de fichiers. Réagit aux créations, modifications ou suppressions de fichiers en temps réel.

smtplib / emails

[Notifications email]

Envoi automatique d'emails. Notifications d'alertes, rapports périodiques, pièces jointes, HTML emails.

openpyxl / python-docx

[Rapports Office]

Génération automatique de rapports Word et Excel. Remplissage de templates, tableaux, graphiques Excel.



Automatisation — Surveillance & Rapport Email Automatique

```
import os, smtplib, schedule, time
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
from email.mime.text import MIMEText

rapport = []

# — ÉTAPE 1 : Surveiller un dossier —————
class MonSurveillant(FileSystemEventHandler):
    def on_created(self, event):
        msg = f"NOUVEAU : {os.path.basename(event.src_path)}"
        rapport.append(msg)
        print(msg)
    def on_modified(self, event):
        msg = f"MODIFIÉ : {os.path.basename(event.src_path)}"
        rapport.append(msg)

# — ÉTAPE 2 : Envoyer rapport par email —————
def envoyer_rapport():
    if not rapport: return
    contenu = "\n".join(rapport)
    msg = MIMEText(f"Rapport automatique :\n\n{contenu}")
    msg["Subject"] = "Rapport de surveillance Python"
    msg["From"] = "python@monserveur.mg"
    msg["To"] = "admin@monserveur.mg"

    with smtplib.SMTP("smtp.gmail.com", 587) as s:
        s.starttls()
        s.login("python@gmail.com", "mot_de_passe")
        s.send_message(msg)
    rapport.clear()
    print("Rapport envoyé !")

# — ÉTAPE 3 : Lancer la surveillance —————
observer = Observer()
```

Watchdog Observer

Détection instantanée de tout changement dans un répertoire.

Schedule

Rapport envoyé automatiquement chaque jour à 18h00.

smtplib Email

Notification email avec résumé des activités de la journée.



Développement de Jeux — Python dans le Gaming



Pygame

Bibliothèque 2D complète. Sprites, sons, événements, collisions. Parfaite pour apprendre le développement de jeux.

→ Jeux 2D, arcade, éducatif

Panda3D

Moteur 3D open-source. Utilisé par Disney. Rendu 3D, physique, shaders, animation.

→ Jeux 3D, simulations

Ren'Py

Moteur de visual novels. Interface narrative, scènes, dialogues, choix. Python pour les scripts d'histoire.

→ Visual novels, éducatif

```
# Mini jeu Pygame – Boucle principale
import pygame
pygame.init()
ecran = pygame.display.set_mode((640, 480))
pygame.display.set_caption("Mon Jeu Python")
clock = pygame.time.Clock()
```

```
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
```



Python & l'Avenir — Tendances 2025-2030



IA Générative

LLMs, Copilots, agents IA. Python est le langage exclusif des modèles GPT, Llama, Gemini. Cette tendance s'accélère.



Edge Computing

MicroPython sur puces ARM Cortex. Python s'étend aux objets connectés, véhicules autonomes, drones.



Automatisation RPA

Remplacement de tâches répétitives. Python + IA = RPA intelligente. Demande en explosion dans les entreprises.



Data Engineering

Pipelines de données (Airflow, dbt, Spark via PySpark). Python est la lingua franca de la data moderne.



WebAssembly (WASM)

Pyodide, PyScript : Python directement dans le navigateur. Web interactif sans JavaScript obligatoire.



Quantique (Qiskit)

IBM Qiskit est écrit en Python. L'informatique quantique accessible via Python dès aujourd'hui.



Ressources & Roadmap — Comment Démarrer ?

Mois 1-2

Fondamentaux

python.org · [Automate the Boring Stuff](#) · [CS50P](#) (Harvard gratuit)

Mois 3-4

Choisir un domaine

Choisir parmi : Web, IA, Data, Réseau, IoT selon vos objectifs

Mois 5-6

Projets réels

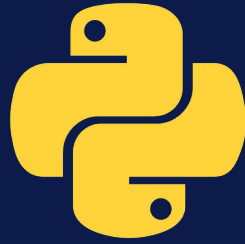
[GitHub](#) · [Kaggle \(data\)](#) · [HackTheBox \(cyber\)](#) · [LeetCode \(algo\)](#)

Mois 7-12

Spécialisation

Certifications : [PCE1 Python](#), [AWS](#), [Google ML](#) · [Portfolio](#) · [Open Source](#)





Technologies Python

**Un seul langage,
infinie de possibilités**

Simple

Polyvalent

Puissant

Du Futur

Python Core + Framework = Expert du Domaine

Web · IA · Data Science · Cybersécurité · Admin Réseau · IoT · SIG · Automatisation · Jeux